Isolating Compiler Faults via Multiple Pairs of Adversarial Compilation Configurations

QINGYANG LI, YIBIAO YANG, MAOLIN SUN, JIANGCHANG WU, QINGKAI SHI, and YUMING ZHOU, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China

Compilers are fundamental to modern software development, making the effective identification and resolution of compiler faults essential. However, localizing these faults to specific source files remains highly challenging due to the complexity and scale of modern compiler infrastructures. Prior approaches that rely on passing executions of randomly generated witness test programs often incur high computational costs and produce unstable localization results. Although recent techniques that derive passing compilation configurations from failing ones have partially alleviated these limitations, substantial room for improvement remains. In this study, we propose MultiConf, a novel approach that automatically isolates compiler faults by constructing multiple pairs of adversarial compilation configurations. Each adversarial compilation configuration pair consists of a failing configuration and its corresponding passing configuration, which differ in only a small number of fine-grained options. MultiConf generates failing configurations through a lightweight construction process and derives the corresponding passing configurations by selectively disabling bug-related fine-grained options. We then employ a Spectrum-Based Fault Localization (SBFL) formula to rank the suspiciousness of compiler source files. Each adversarial configuration pair independently produces a ranking, which is subsequently aggregated using a weighted voting scheme to derive a final suspiciousness ranking, enabling more accurate and robust fault localization. We evaluate MultiConf on a benchmark of 60 real-world GCC compiler bugs. The results demonstrate that MultiConf significantly outperforms existing compiler fault localization techniques in both effectiveness and efficiency. In particular, MultiConf successfully localizes 27 out of 60 bugs at the Top-1 file level, representing improvements of 35.0% and 28.6% over the two state-of-the-art approaches, Odfl (20) and Basic (21), respectively.

CCS Concepts: • **Software and its engineering** → *Compilers*; **Software testing and debugging**;

Additional Key Words and Phrases: Compilers, Fault Localization, Adversarial configuration, Multiple

This work was supported in part by the National Natural Science Foundation of China (Grants 624B2067, 62572226, and 62472215), the Jiangsu Natural Science Foundation (Grant BK20231402), and the Collaborative Innovation Center of Novel Software Technology and Industrialization.

Authors' Contact Information: Qingyang Li, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: liqingyang@smail.nju.edu.cn; Yibiao Yang (corresponding author), State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: yangyibiao@nju.edu.cn; Maolin Sun, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: merlin@smail.nju.edu.cn; Jiangchang Wu, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: jiangchangwu@smail.nju.edu.cn; Qingkai Shi, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: qingkaishi@nju.edu.cn; Yuming Zhou (corresponding author), State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: zhouyuming@nju.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7392/2026/9-ART285
https://doi.org/10.1145/3786786

ACM Reference format:
Qingyang Li, Yibiao Yang, Maolin Sun, Jiangchang Wu, Qingkai Shi, and Yuming Zhou. 2026. Isolating Compiler Faults via Multiple Pairs of Adversarial Compilation Configurations. *ACM Trans. Softw. Eng. Methodol.* 35, 9, Article 285 (September 2026), 30 pages.
https://doi.org/10.1145/3786786

1 Introduction

Compilers play a crucial role in software development by translating high-level programming code into executable machine code. However, the compilation phase can introduce bugs, resulting in compiler faults [36]. Such faults may cause incorrect program outputs, system crashes, or even security vulnerabilities. These issues pose serious challenges to software reliability and security, making both the compiler testing and fault localization crucial for trustworthy software systems.

Compiler testing identifies the presence of bugs [47], while fault localization (the focus of our work) aims to pinpoint the specific source files or components that are responsible for these bugs [10]. A large body of research has focused on compiler testing (i.e., automatically finding scenarios where compilers misbehave) [15, 22, 45, 47]. Researchers have proposed diverse approaches for compiler bug detection, as surveyed by Chen et al. [12]. These include random program generation [47], differential testing [28, 30, 34, 50], and metamorphic testing [23, 24], which have significantly improved the ability to uncover subtle compiler faults. Notably, recent works have explored compiler configuration spaces to boost bug detection effectiveness, such as exploring various optimization settings [13], searching two-dimensional input spaces for JIT compilers [19], attribute-guided compilation space exploration [45], and auto-tuning via critical flag selection [56]. Other research has focused on detecting missed optimizations [7] and warning defects [35].

Once a bug is detected, the next challenge is fault localization: determining which specific source files are responsible. This task is non-trivial, given the intricate and extensive nature of modern compilers. To illustrate the scale of the challenge, consider the GCC compiler: the average number of potentially relevant C source files per bug scenario is 380 (i.e., .c files in the top-level/trunk/gcc directory, excluding subdirectories), while the entire codebase contains more than 1,600 .c, .cc, and .h files (recursively searched in /trunk/gcc, excluding test files). This considerable scale highlights the practical difficulty of pinpointing the faulty component and motivates the need for more efficient and accurate fault localization methods.

Several approaches have been proposed for compiler fault localization. Witness test program-based methods, such as DiWi [10], RecBi [11], LLM4CBI [39], and ETEM [25], rely on generating passing and failing test programs to isolate faults. A failing test program is specifically designed to expose a compiler bug, whereas a passing test program is expected not to trigger the same bug under the identical compilation configuration. With these test programs, the coverage spectra can be obtained from the corresponding execution traces. Subsequently, the **Spectrum-Based Fault Localization (SBFL)** techniques can be applied to calculate the suspiciousness scores of code entities in compilers [4, 38]. However, as pointed out by Yang et al. [49], such approaches suffer from inherent limitations, including high computational overhead and instability due to randomness in mutation process. To address these limitations, Yang et al. proposed an alternative approach, ODFL [49], which constructs a single pair of adversarial compilation configurations to efficiently generate passing and failing executions for SBFL-based fault localization.

While ODFL demonstrates improved accuracy, it considers only a single pair of adversarial configurations. This design choice limits its ability to fully exploit diverse coverage information, as the effectiveness of SBFL critically depends on both the quality and the quantity of the test executions [21, 46]. Motivated by ensemble learning, where aggregating multiple models typically

yields more robust and accurate performance [54, 55], we observe that leveraging multiple pairs of adversarial compilation configurations can provide complementary diagnostic information. In practice, different strategies can be employed to obtain multiple pairs of adversarial compilation configurations, as different pairs possess varying capabilities for isolating irrelevant code elements. For instance, consider a compiler consisting of four files: A, B, C, and D, where D is the faulty file responsible for a bug. One configuration pair may isolate innocent files A and B, while another may isolate innocent files B and C. In isolation, neither pair can accurately identify the truly faulty file D. However, by aggregating the innocent files identified across these pairs, we can effectively infer that D is responsible for the bug. Consequently, generating multiple pairs of adversarial compilation compilations can enhance the ability to isolate compiler faults.

Approach. In this study, we propose MultiConf, a novel method for isolating compiler faults by aggregating individual rankings derived from multiple pairs of adversarial compilation configurations. Our key insight is that different pairs possess varying capabilities for isolating irrelevant code responsible for compiler bugs. By aggregating the SBFL ranking results from these diverse pairs, we achieve more accurate localization results that surpass what any individual pair can provide in isolation. We employ various strategies to generate multiple pairs of adversarial compilation configurations, ensuring that each pair maintains internal similarity. This strategic combination enables us to effectively leverage the strengths of each configuration pair, resulting in improved fault localization.

Given the significant number of source files within the compiler, our initial step is to narrow down the pool of potential faulty files using execution traces. To generate adversarial compilation configurations, we systematically disable each fine-grained option and identify bug-triggering options that convert a failing configuration into a passing one. These bug-triggering options represent potential root causes of the bug. Next, we generate multiple pairs of adversarial compilation configurations by selecting different failing configurations. Each failing configuration produces corresponding passing configurations by selectively disabling the identified bug-triggering options. Subsequently, we employ SBFL techniques to calculate the suspiciousness levels of compiler files for each configuration pair. This analysis enables us to rank candidate faulty compiler files based on their suspiciousness values, providing a measure of their likelihood of containing faults. Finally, we prioritize files that consistently appear as suspicious across the majority of rankings using a weighted voting system. By aggregating these rankings, we can identify potentially faulty files.

Our evaluation of MultiConf using 60 real GCC bugs demonstrates its effectiveness in localizing compiler bugs. In the Top-1, Top-5, Top-10, and Top-20 files of the ranking lists, MultiConf successfully localizes 27, 40, 51, and 53 compiler bugs out of the total 60 GCC bugs, respectively. The most significant improvement is observed in Top-1 accuracy, where MultiConf achieves an 35.0% enhancement over Odfl, the state-of-the-art SBFL technique (localizing 27 bugs versus Odfl's 20). This advancement is particularly valuable for real-world debugging, as developers typically inspect the highest-ranked suspicious files. Given the average of 380 source files considered per bug, these results highlight the practical utility and robustness of our multi-pair strategy.

Contributions. We make the following main contributions:

- *Methodology*: We introduce a systematic procedure for constructing multiple pairs adversarial compilation configurations, each consisting of a failing configuration and a corresponding passing configuration with adversarial relationships. The procedure selectively disables fine-grained options within failing configurations to identify options that potentially trigger compiler bugs.
- *Originality*: We propose a novel approach that strategically leverages multiple pairs of adversarial compilation configurations and aggregates multiple ranking results to precisely

pinpoint compiler faults. To the best of our knowledge, we are the first to leverage multiple pairs of adversarial compilation configurations for compiler fault localization.
— *Implementation*: We implement our approach as a practical fault localization tool, MultiConf, which is powered with multiple pairs of adversarial compilation configurations. MultiConf provides a lightweight yet effective solution for localizing compiler faults in practice.
— *Evaluation*: We conduct extensive empirical evaluations of MultiConf alongside existing techniques on a benchmark of 60 real-world GCC compiler bugs. The results demonstrate substantial improvements in Top-1 localization accuracy, highlighting the practical effectiveness of our approach for identifying faults in GCC.

Organization. The rest of this article is organized as follows. Section 2 introduces the background and presents our motivation of leveraging multiple pairs of adversarial compilation configurations. Section 3 elaborates on our approach in detail. Then, our experimental setup and results are presented in Section 4. After that, we discuss the limitations, threats to the validity, and future work in Section 5. Finally, Section 6 surveys related work, and Section 7 concludes this article.

2 Background and Motivation

In this section, we first briefly describe the SBFL technique utilized in our study. Then, we introduce the background of the compiler options and compiler faults. Finally, we summarize existing techniques in locating compiler faults, discuss their limitations, and offer a clear and concise explanation of the motivation behind our approach.

2.1 SBFL

SBFL has received significant attention in the research community as a debugging technique to identify the root cause of software faults or failures [4, 46]. It leverages coverage information from executed code to assess the suspiciousness of individual code statements or blocks, subsequently ranking them based on their likelihood of being faulty [4]. This approach has been extensively studied in academic literature and is recognized for its potential to guide fault localization in various scenarios. The fundamental principle of SBFL is based on the assumption that code statements or blocks executed more frequently by failing test cases are more likely to contain faults, while those executed more frequently by passing test cases are less likely to be faulty. By ranking code elements according to their suspiciousness scores, SBFL provides developers with a prioritized list of code locations to examine, potentially reducing the time and effort required for debugging. However, the effectiveness of SBFL heavily depends on the quality and quantity of test executions [21, 46]. In particular, a lack of diverse failing test cases or inadequate representation of the fault in test cases can compromise its accuracy. A previous study [21] has demonstrated that increasing the number of failing test cases enhances the effectiveness of SBFL by providing more diverse coverage information, which is critical for accurate fault localization. Despite its success in research settings, there is limited evidence to suggest that SBFL has been widely adopted in the software industry. Its application in practice may be constrained by challenges such as the need for comprehensive test suites and the high computational cost of analyzing complex systems. Nonetheless, its potential to assist in debugging, as shown in research studies, continues to drive interest in further refinement and application of SBFL techniques.

2.2 Compiler Faults

Compiler faults are errors within the compiler that can cause incorrect program execution or unexpected behavior. Notably, an empirical study of compiler bug reports revealed that optimization faults constitute the largest proportion of reported compiler issues [53]. This prevalence underscores

the significant impact of optimization-related bugs on software reliability and maintainability. These faults often stem from the complexity of compiler optimization algorithms and their intricate interactions with source code, making them difficult to detect and fix. Given their frequency and the critical role of compilers in software development, there is a clear need for effective techniques to localize and address optimization faults. This motivates our study, which focuses on improving fault localization specifically for fine-grained compiler optimization options.

2.3 Localization of Compiler Faults

Localizing compiler faults is inherently challenging due to the complexity and scale of modern compiler infrastructures. During the code generation phase, compilers apply numerous optimizations whose intricate interactions can introduce subtle and hard-to-diagnose bugs [47]. Diagnosing and localizing optimization-related bugs is difficult because of the combinatorial explosion of possible optimization configurations, making manual debugging both time-consuming and error-prone.

To address these challenges, a variety of automated compiler fault localization techniques have been proposed. From the perspective of how passing executions are constructed to contrast with failing ones, existing approaches can be broadly classified into two categories: program mutation-based localization and compilation configuration-based localization.

— *Program Mutation-Based Localization*: These approaches generate passing executions by mutating the original bug-triggering programs to produce witness programs that no longer trigger the bug. By comparing the compiler execution traces of failing programs and their corresponding witness programs, SBFL techniques can be applied to identify suspicious fault locations. Representative techniques include DiWi [10], which synthesizes witness programs using local operation mutations combined with Metropolis-Hastings sampling; RecBi [11], which employs reinforcement-learning-guided structural mutations; LLM4CBI [39], which leverages **Large Language Models (LLMs)** to generate effective witness programs; and ETEM [25], which proposes enhanced mutation strategies, simplified optimization configurations, and improved suspiciousness formulas for file-level localization.
— *Compilation Configuration-Based Localization*: Instead of modifying test programs, this line of work generates passing executions by manipulating compiler optimization configurations. ODFL [49], for example, systematically disables fine-grained optimization options in a failing compilation configuration to construct adversarial passing–failing configuration pairs for the same test program. By contrasting compiler behaviors under these configurations, fault localization can be effectively performed.

Program mutation-based approaches rely on the availability of both failing and passing test programs. While bug-triggering programs are designed to expose specific compiler faults, witness programs are crafted to avoid triggering the bug while remaining semantically similar. The resulting contrast between passing and failing executions enables SBFL techniques to identify potential faulty components within the compiler. In contrast, compilation configuration-based approaches operate on a fixed test program and instead explore the optimization configuration space. For a given program, some configurations lead to failing compilations, whereas others succeed. By systematically identifying such adversarial configuration pairs, these approaches similarly obtain sufficient execution diversity to support SBFL-based fault localization, without incurring the substantial cost associated with program-level mutations.

2.4 Discussion of Existing Techniques

Existing techniques have demonstrated effectiveness in locating compiler bugs. However, as pointed out by Yang et al. [49], witness test program-based approaches, such as DiWi [10], RecBi [11],

```c
int a;
int b;
int *c = &a;
unsigned short d;

int main () {
  unsigned int e = a;
  *c = 1;
  if (!b) {
      d = e;
      *c = d | e;
    }
  if (a != 0) {
    __builtin_abort ();
  }
  return 0;
}
```

(a) Bug-triggering test program g.c

```
$ gcc -O1 g.c && ./a.out
$ gcc -Os g.c && ./a.out
Aborted (core dumped)
$ gcc -Os -fno-expensive-optimizations g.c & ./a.out
```

(b) Compile configurations

Fig. 1. GCC bug #61517. When the test program is compiled at the "-O1" optimization level, it executed as expected and did not produce any output. While at the "-Os" optimization level, the program aborted and generated a core dump.

Table 1. Statistics of the Fine-Grained Optimization Options Enabled by Default at the Lowest Bug-Triggering Optimization Level for the Bugs in the Benchmark

Bug-Triggering Level	Minimum	Maximum	Median	Mean
Lowest	78	141	119	116

LLM4CBI [39], and ETEM [25], can be time-consuming and unstable due to the inherent randomness in the mutation process.

The adversarial configuration-based approach ODFL [49] has demonstrated that the optimization processes associated with some particular options may serve as the root cause of bugs and passing configurations can be generated by disabling these bug-triggering options. As shown in Figure 1, disabling specific fine-grained optimization options in GCC can effectively hide compiler optimization bugs that are initially exposed by the bug-triggering test program.

However, the compilation configuration space is extensive, with a size of 2^n, where n represents the number of fine-grained compilation options that are enabled by default at the bug-triggering level. In most cases, n exceeds 100 (as shown in Table 1), making it challenging to select appropriate configurations for analysis. While some configurations may accurately identify faulty files, others may not, resulting in suboptimal performance of the generated configurations. ODFL is limited by its reliance on a single pair of adversarial compilation configurations, which may fail to capture sufficient coverage diversity for optimal localization performance. Although ensuring similarity among these configurations is essential for distinguishing innocent files from suspicious ones, this focus also restricts the exploration of a broader range of configurations, potentially compromising the accuracy of bug localization results.

On one hand, achieving comprehensive coverage requires expanding the range of adversarial compilation configurations. On the other hand, maintaining similarity among configurations is crucial for more targeted analysis, as similar execution traces enable a more precise assessment of code coverage changes, which helps in distinguishing suspicious files. Initially, these requirements may seem contradictory. To address this challenge, we draw inspiration from ensemble learning,

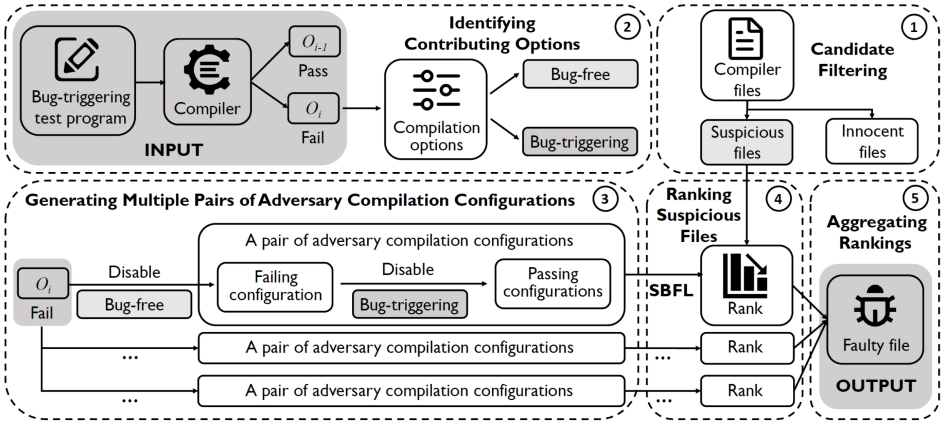


Fig. 2. An overview of MultiConf.

which combines the predictions of multiple base learners to create a more powerful and robust model. We generate multiple pairs of adversarial compilation configurations that maintain similarity within each pair while differing among pairs. By aggregating the localization results obtained from these pairs, we achieve a more robust and reliable outcome for fault localization.

3 Approach

Based on the insights obtained in Section 2, we introduce MultiConf, a novel approach for localizing compiler faults, which leverages multiple pairs of adversarial compilation configurations. In this section, we present an overview of MultiConf, provide a comprehensive explanation of our approach, and describe the SBFL formula employed by MultiConf.

3.1 Overview

We present MultiConf, a novel five-phase methodology for compiler fault isolation that significantly advances the Odfl framework in both effectiveness and efficiency. Figure 2 provides an illustration of our approach.

Although inspired by Odfl and conceptually a multi-pair generalization, MultiConf introduces several essential architectural and algorithmic innovations. Specifically, MultiConf evolves the single-pair analysis of Odfl into a robust multi-pair framework, incorporates novel candidate filtering and weighted aggregation mechanisms, and achieves substantial practical efficiency improvements. These enhancements yield marked gains in fault localization accuracy and robustness, particularly for challenging compiler bugs (see Tables 4 and 7).

MultiConf takes four key inputs: (1) the compiler, (2) a bug-triggering test program, (3) the lowest optimization level that triggers the bug, and (4) the highest optimization level where the bug does not occur. Leveraging these inputs, MultiConf systematically constructs adversarial configuration pairs to guide fault localization. The main output is a ranked list of suspicious compiler source files, along with the identification of optimization options critical to bug manifestation—providing actionable diagnostic insights for developers. The core workflow, outlined in Algorithm 1, proceeds through the following phases.

(1) *Candidate Filtering*: Unlike Odfl, which considers all compiler files as possible fault locations, MultiConf applies a two-stage, coverage-driven filtering to greatly reduce the search space

Algorithm 1: Compiler Faults Isolation with MultiConf

Input: O_{fail}: The lowest optimization level reproducing the bug;
O_{pass}: The highest optimization level concealing the bug.
Output: *Rank*: The aggregated rank of suspicious compiler files.

```
   /* Step1: Candidate Filtering                                          */
1  FailCovSet ← CollectCoverage (O_fail)
2  PassCovSet ← CollectCoverage (O_pass)
3  SuspiciousCovSet ← FailCovSet − PassCovSet
4  SuspiciousFiles ← GetSuspiciousFiles (SuspiciousCovSet)
   /* Step2: Identifying Contributing Options                             */
5  PassResult ← GetResult (O_pass)
6  F ← GetEnabledFineGrainedOptions (O_fail)
7  F_bt ← ∅ /* F_bt: the set of bug-triggering options                   */
8  foreach f ∈ F do
9      conf ← DisableOptions (O_fail, {f})
10     if GetResult (conf) == PassResult then
11         F_bt ← F_bt ∪ {f}
12 F_bf ← F − F_bt /* F_bf: the set of bug-free options                   */
   /* Step3: Generating Adversarial Configurations                        */
13 AdvConf_1 ← GenAdvConfs (O_fail, F_bf, F_bt)
14 AdvConf_2 ← GenAdvConfs (O_fail, GetHalfElement (F_bf), F_bt)
15 AdvConf_3 ← GenAdvConfs (O_fail, ∅, F_bt)
   /* Step4: Ranking Suspicious Files                                     */
16 Rank_1 ← SBFL (AdvConf_1, SuspiciousFiles)
17 Rank_2 ← SBFL (AdvConf_2, SuspiciousFiles)
18 Rank_3 ← SBFL (AdvConf_3, SuspiciousFiles)
   /* Step5: Aggregating Rankings                                         */
19 Rank ← RankAggr (Rank_1, Rank_2, Rank_3)
20 return Rank

21 Function GenAdvConfs(O_fail, F_disable, F_bt)
       /* F_disable: the selected bug-free options from F_bf              */
22     FailConf ← DisableOptions (O_fail, F_disable)
23     PassConfs ← ∅
24     foreach F_bt ∈ F_bt do
25         conf ← DisableOptions (FailConf, {F_bt})
26         PassConfs ← PassConfs∪ {conf}
       /* FailConf: the failing configuration                             */
       /* PassConfs: a set of passing configurations created by disabling each bug-triggering
          option in FailConf                                              */
27     return (FailConf, PassConfs)
```

(Lines 1–4). This step narrows the candidate set by 66% on average (from 380 to 129 files), improving both efficiency and precision.

(2) *Identifying Contributing Options*: Like Odfl, MultiConf identifies fine-grained optimization options directly involved in triggering the bug (Lines 5–12).

(3) *Generating Multiple Adversarial Configurations*: In contrast to ODFL's single-pair approach, MULTICONF systematically constructs multiple adversarial configuration pairs from distinct failing configurations (Lines 13–15). Each passing configuration in the pair is created by independently disabling a bug-triggering option. This strategy captures a wider spectrum of failure-inducing conditions. In our experiments, we generate three such pairs per bug instance.
(4) *Ranking Suspicious Files*: Each adversarial pair undergoes SBFL analysis, as in ODFL, producing multiple independent suspiciousness rankings (Lines 16–18). This multi-perspective analysis increases localization robustness, as different pairs may highlight different aspects of the underlying fault.
(5) *Aggregating Rankings*: MULTICONF introduces a novel weighted voting scheme to aggregate rankings from all adversarial pairs (Line 19). Files consistently ranked as highly suspicious across pairs receive higher priority, yielding a more reliable and discriminative identification of faulty code regions.

MULTICONF achieves substantial advancement over prior work in three key areas.

— *Architectural Generalization*: MULTICONF systematically integrates diagnostic evidence from multiple configuration pairs, addressing the instability and coverage limitations inherent to single-pair approaches like ODFL. This architecture not only improves accuracy in fault localization but also enables new forms of analysis that are not possible with single-pair methods.
— *Algorithmic Contributions*: Beyond the multi-pair paradigm, MULTICONF introduces two novel algorithmic components: candidate filtering and weighted aggregation. These innovations optimize both the efficiency and the discriminative power of the fault localization process, and can be generalized to other configuration-based or program-based localization frameworks.
— *Practical Improvement*: MULTICONF features an improved adversarial configuration generation strategy, yielding a 93% reduction in generation time compared to ODFL (see Table 7). This makes the approach more practical and scalable for real-world use.

Overall, MULTICONF offers clear advantages over ODFL by providing (1) robust performance across diverse configuration pairs and (2) effective exploitation of varied coverage patterns from multiple configurations. Experimental results demonstrate that MULTICONF maintains ODFL's computational efficiency while delivering substantially higher localization accuracy, particularly for complex optimization bugs where traditional single-pair approaches are less effective.

3.2 Key Steps

To isolate compiler faults, MULTICONF employs a five-step process that involves multiple pairs of adversarial compilation configurations. We provide a detailed outline of each key step below.

3.2.1 Candidate Filtering. We begin by defining the candidate scope and "clean" files for each bug. Candidates include all C source files in the main GCC directory (i.e., *.c files under gcc), averaging 380 files per bug. The faulty file(s) are those modified by the developer when fixing the bug; "clean" files are all remaining files.

We then reduce the candidate set via two filtering steps (Lines 1–4), summarized in Table 2.

— *Coverage Presence Filter*: We remove files that are not executed when compiling the bug-triggering program under the bug-triggering configuration. The remaining "Files Covered" retain only executed files relevant to the compilation.

Table 2. Statistic of the Decreased Number of Suspicious Files after Filtering for the Bugs in the Benchmark

	Average	⇓	Maximum	⇓	Minimum	⇓
Compiler Files	380	—	433	—	359	—
Files Covered	304	20.00	345	20.32	169	52.92
Files with Diff Cov	129	66.05	197	54.50	52	85.52

Columns "⇓" refers to the decreased rate after filtering.

— *Differential Spectrum Filter*: We compare spectra between the failing and passing configurations and remove files whose coverage is identical across the two, as they provide no discriminative signal for SBFL. The remaining "Files with Diff Cov" are those with differing spectra across the initial adversarial pair.

Across the benchmark, these two steps reduce the average candidate set from 380 to 129 files, substantially shrinking the search space without excluding any true faulty files in our dataset. This improves both the efficiency and the robustness of subsequent SBFL and aggregation.

3.2.2 Identifying Contributing Options. The second step of our approach aims to identify the specific fine-grained options that directly contribute to the occurrence of the bug. This step is motivated by the observation that most bugs can be concealed when lower levels of coarse-grained optimization or no optimization are applied. Therefore, it is reasonable to assume that the occurrence of optimization bugs is associated with certain fine-grained options.

To facilitate further analysis and discussion, we introduce the following definitions.

— *Bug-Triggering Option*: A fine-grained option is categorized as bug-triggering if disabling it at the bug-triggering optimization level conceals the bug.
— *Bug-Free Option*: A fine-grained option is categorized as bug-free if disabling it at the bug-triggering optimization level still triggers the bug.

To effectively narrow down the search space, we begin by selecting the lowest coarse-grained optimization level that reproduces the bug. For instance, if "-O0" conceals the bug while both "-O1" and "-O2" reproduce it, we select "-O1". This approach minimizes the number of enabled fine-grained optimizations. We then systematically disable each fine-grained optimization option individually (Line 9). By observing the program's behavior and output, we can identify the bug-triggering options. If disabling a specific option successfully conceals the bug—meaning the program's behavior and output align with expectations—we categorize that option as a bug-triggering option (Lines 10–11). The remaining options are classified as bug-free options (Line 12). This categorization process allows us to pinpoint the fine-grained optimizations directly responsible for triggering the bug and facilitates further analysis.

3.2.3 Generating Adversarial Configurations. In the third step of our approach, our objective is to generate multiple pairs of adversarial compilation configurations using the bug-triggering and bug-free options identified in the previous step. This process allows us to examine the specific contributions of these optimizations to the bug from different perspectives.

For a specific test program, different compiler setups can yield varying results: some configurations (failing configurations) trigger the bug, while others (passing configurations) do not. These configurations are considered adversarial because they exhibit distinct behaviors with respect to the bug. Our goal is to construct adversarial configurations that differ by only a few options.

In our approach, the GenAdvConfs procedure produces an $AdvConf$ pair, comprising a failing configuration and a set of passing configurations. Formally, each pair is denoted as ($FailConf$, $PassConfs$), where $FailConf$ is a configuration that manifests the bug, and $PassConfs$ is a collection of configurations that do not exhibit the bug. Each passing configuration in $PassConfs$ is derived by disabling exactly one bug-triggering option present in the failing configuration, ensuring that it differs from $FailConf$ by only a single option (Lines 24–26). Consequently, every ($FailConf$, $PassConfs$) pair forms an adversarial relationship suitable for SBFL analysis.

For this study, we select as our initial failing configuration the lowest optimization level at which the bug can be reproduced. We then generate three distinct types of failing configurations by disabling different subsets of bug-free options, as follows.

— $FailConf_1$: Disables all bug-free options (Line 13).
— $FailConf_2$: Disables half of the bug-free options (Line 14).
— $FailConf_3$: Disables none of the bug-free options (Line 15).

By constructing these failing configurations, GenAdvConfs yields three corresponding adversarial pairs, ($FailConf$, $PassConfs$), denoted as $AdvConf_1$, $AdvConf_2$, and $AdvConf_3$. In each case, the passing configurations in $PassConfs$ are adversarial with respect to their associated $FailConf$. This systematic exploration enables us to thoroughly investigate the influence of bug-triggering options and to gain deeper insights into their role in exposing the bug.

While our algorithm effectively identifies directly causative bug-triggering options, there are cases in which no such options can be isolated. This situation arises either (1) when the bug results from inherent behavior at a specific optimization level, rather than from individual fine-grained options, or (2) when the bug emerges only due to complex interactions among multiple options. To address these scenarios, we additionally generate adversarial configuration pairs at the optimization level granularity. In these cases, each pair ($AdvConf$) is of the form (O_{fail}, O_{pass}), where O_{fail} denotes the bug-triggering optimization level and O_{pass} is a lower, non-bug-triggering optimization level.

3.2.4 Ranking Suspicious Files. In the fourth step of our approach, we employ the SBFL technique to localize faults within each pair of collected adversarial compilation configurations. SBFL is based on the assumption that code statements executed frequently in different passing cases are less likely to be the cause of the fault, while specific code statements executed due to the enabling of certain bug-triggering options are more likely to be the source of the issue. Following the approach of previous studies [10, 11, 49], our tool, MultiConf, utilizes the Ochiai formula [2] to calculate the suspiciousness value for each statement s that is executed or not executed within the configurations. The Ochiai formula, defined in Formula (1), takes into account the number of bug-triggering cases (ef_s) that execute the statement s, the number of bug-concealing cases (ep_s) that execute the statement s, and the number of bug-triggering configurations (nf_s) that do not execute the statement s:

$$suspicious(s) = \frac{ef_s}{\sqrt{(ef_s + nf_s)(ef_s + ep_s)}}. \tag{1}$$

In this study, since all passing configurations are paired with the same failing configuration that triggers the bug, and we are specifically interested in the statements executed by the failing case, we can simplify the formula as follows:

$$suspicious(s) = \frac{1}{\sqrt{1 + ep_s}}. \tag{2}$$

After computing the suspiciousness values for each affected statement in the suspicious compiler files identified in the first step, MultiConf proceeds to calculate the overall suspiciousness value

for each of these files. This is achieved by aggregating the suspiciousness values of all statements within the file, using the aggregation formula established in previous studies [10, 11, 49] as shown in Formula (3). Here, n_f denotes the count of statements covered by the failing configuration in the compiler file f:

$$suspicious(f) = \frac{\sum_{i=1}^{n_f} suspicious(s_i)}{n_f}. \tag{3}$$

Finally, the compiler files are ranked based on their suspiciousness values, with higher-ranked files indicating a greater probability of containing bugs.

3.2.5 Aggregating Rankings. The final step of our approach involves aggregating the three individual ranks obtained from the previous step to produce a consolidated ranking of the compiler files. To achieve a more robust and reliable evaluation of the suspiciousness of these files, we prioritize those that consistently appear as suspicious across the majority of individual rankings, using a weighted voting system.

The weighting of the individual rankings is a crucial component of this aggregation process. In the context of compiler fault localization, the Top-N metric is a widely recognized evaluation criterion, where files ranked higher are considered more impactful and deserving of a development team's limited time and resources. These higher-ranked files typically become the focus of further investigation and debugging efforts, as they are more likely to reveal the root causes of the observed compiler issues.

We recognize that the original suspicious scores used to generate individual rankings may vary or exhibit inconsistencies across different ranking methods. This is why we utilize multiple pairs of adversarial compilation configurations to produce varied ranks. Such discrepancies can stem from the unique characteristics and assumptions of each ranking approach, as well as the inherent complexity of the compiler codebase. To address these potential differences, we base the voting power of each file on its rank value rather than solely on the raw suspicious scores.

Specifically, we assign the following voting powers to files based on their rank positions.

- —Files ranked at the very top (e.g., rank 1) receive the highest voting power, such as 5 points.
- —Files within the Top-5 range receive 4 points.
- —Files in the Top-10 range receive 3 points.
- —Files in the Top-20 range receive 2 points.
- —For files ranked outside the Top-20, we use the multiplicative inverse of their rank value as the voting power (e.g., a file ranked 50th would have a voting power of $\frac{1}{50} = 0.02$).

After determining the individual voting powers, we sum the three voting scores for each file to arrive at the final aggregated ranking. This rank-based weighting scheme ensures that files consistently ranked at the top positions, regardless of their specific suspicious score values, have the most influence in the final aggregated ranking.

Case Study: GCC Bug #70138. Consider GCC bug #70138, involving two files of interest: `tree-vect-slp.c` and `tree-vect-loop-manip.c`, each evaluated across three adversarial configuration pairs.

- —`tree-vect-slp.c`: ranks 1, 1, 67 (scores 0.707, 0.707, 0.411).
- —`tree-vect-loop-manip.c`: ranks 4, 3, 4 (scores 0.655, 0.647, 0.701).

Computing aggregate scores:

$$V_{\texttt{tree-vect-slp.c}} = 5\,(\text{rank } 1) + 5\,(\text{rank } 1) + 1/67\,(\text{rank } 67) \approx 10.015$$
$$V_{\texttt{tree-vect-loop-manip.c}} = 4\,(\text{rank } 4) + 4\,(\text{rank } 3) + 4\,(\text{rank } 4) = 12.$$

Despite `tree-vect-slp.c` attaining first place in two analyses, its aggregate score is reduced due to the outlier 67th rank. In contrast, `tree-vect-loop-manip.c` achieves a higher aggregate due to its stable Top-5 performance.

Analysis. Manual inspection confirms the true bug resides in `tree-vect-loop-manip.c`, not in `tree-vect-slp.c`. The latter's sporadically high ranks are attributed to coincidental coverage, illustrating the danger of over-reliance on single-configuration results.

Our aggregation scheme offers three key advantages.

(1) *Outlier Mitigation*: Files with highly inconsistent ranks are downweighted.
(2) *Stability Reward*: Files consistently appearing in top positions accumulate higher scores.
(3) *Robustness*: The final ranking reflects consensus across diverse analyses, reducing the impact of configuration-specific artifacts.

In summary, rank-based aggregation enables more reliable identification of faulty files by amplifying consistent signals and attenuating noise, thus supporting more effective fault localization in complex compiler codebases.

4 Evaluation

In this study, we address the following **Research Questions (RQs)**:

— *RQ1*: How effective is MultiConf in locating compiler faults?
— *RQ2*: How efficient is MultiConf in locating compiler faults?
— *RQ3*: What is the impact of candidate filtering on the effectiveness of MultiConf?
— *RQ4*: How does the number of adversarial compilation configuration pairs influence the performance of MultiConf?
— *RQ5*: How do different SBFL formulae affect the performance of MultiConf?

RQ1 evaluates the effectiveness of MultiConf in isolating compiler faults compared to existing techniques. Through controlled experiments on 60 real GCC bugs, we quantitatively assess MultiConf's accuracy in Top-N fault localization metrics (N = 1,5,10,20) and compare with state-of-the-art approaches (Odfl, DiWi, RecBi, LLM4CBI, and ETEM). This investigation aims to demonstrate MultiConf's capability to precisely identify faulty files in compiler.

RQ2 focuses on evaluating the efficiency of MultiConf in locating compiler faults. Specifically, we aim to investigate the computational resources required, particularly the execution time needed to generate multiple pairs of compilation configurations for fault localization. By assessing the efficiency of MultiConf, we aim to determine whether it can provide time-efficient localization of compiler faults.

RQ3 examines the impact of candidate filtering on localization accuracy. Through ablation studies, we analyze how the two-stage filtering—first removing non-executed files, then eliminating files with identical coverage spectra—reduces the search space from 380 files to 129 files (66% reduction) while preserving true faulty files.

RQ4 explores the impact of the number of adversarial configuration pairs on localization performance. We systematically vary the pair count (1, 3, 5, 10) while measuring: (1) Top-N accuracy, (2) ranking stability (**Mean First Rank (MFR)/Mean Average Rank (MAR)**), and (3) computational cost. This reveals the optimal balance between diversity benefits and resource overhead.

RQ5 assesses MultiConf's robustness across six SBFL formulae (Ochiai, Tarantula, DStar, etc.). Through controlled experiments, we measure the variance in Top-N accuracy, confirming that MultiConf's performance is stable regardless of the underlying SBFL formula choice.

Table 3. Distribution of Bug Types in the Benchmark

Bug Type	Count
Wrong Code	55
ICE on Valid Code	5
Total	60

4.1 Experimental Setup

4.1.1 Benchmark. Our research focuses on GCC, a widely used open source C compiler [1], which has become a standard in both industry and academia due to its extensive application across diverse domains [8, 9, 14, 16]. To evaluate the effectiveness of MultiConf, we employ a benchmark comprising 60 real-world GCC bugs, consistent with prior studies. Each bug in this benchmark represents a confirmed issue that has been fixed by developers. The benchmark, originally released by RecBi [11] and subsequently adopted by Odfl [49], LLM4CBI [39], and ETEM [25], enables reliable and fair comparisons with prior work.

For each of the 60 GCC bugs, we collected essential information from the corresponding bug reports: the lowest optimization level that triggers the bug, the minimal test program that reproduces it, the specific GCC version containing the bug, and the compiler source files responsible for the faulty behavior. This curated dataset serves as the ground truth for evaluation. We assess MultiConf's localization ability by querying the ordinal position of the true faulty files in MultiConf's ranking: higher ranks (smaller numbers) indicate better localization.

We intentionally limit our evaluation to GCC rather than including LLVM to ensure a meaningful and challenging assessment. LLVM's strictly ordered pass pipeline enables efficient binary search to locate bug-triggering passes; the identified pass typically aligns with the best failing configuration for fault localization, making multiple adversarial configuration pairs less impactful. In contrast, GCC exposes a richer, more challenging space: its many fine-grained, unordered options make it difficult to identify strong adversarial configurations. This setting better stresses and differentiates approaches like MultiConf.

We summarize the bug types in our benchmark in Table 3. Among the 60 bugs, 55 are wrong-code bugs and 5 are **Internal Compiler Error (ICE)** on valid code.

4.1.2 Baselines. To evaluate the effectiveness of MultiConf in isolating compiler faults, we include representative compilation configuration-based and program mutation-based localization approaches as the baselines.

- *Odfl*: Odfl [49] manipulates fine-grained optimization options to construct a single pair of adversarial compilation configurations, namely passing and failing compilation configurations for the same test program. This enables the collection of contrasting coverage spectra of compilers for SBFL-based analysis.
- *DiWi*: DiWi [10] localizes compiler bugs by applying local operation mutations and employing the Metropolis-Hastings algorithm to sample witness programs.
- *RecBi*: RecBi [11] applies structural mutation techniques to generate witness programs and utilizes a reinforcement learning strategy to guide the search for effective witness test programs.
- *LLM4CBI*: LLM4CBI [39] leverages LLMs to generate effective witness programs for compiler bug localization.

— *ETEM*: ETEM [25] proposes an enhanced program mutation strategy and further adopts adversarial optimization configurations by ODFL together with improved suspiciousness formulas for computing the fault likelihood of files.
— *BASIC*: BASIC [48] identifies the most recent good release and the earliest bad release and applies a binary search to pinpoint the **Bug-Inducing Commit (BIC)**. Files modified in the identified BIC are then regarded as faulty candidates.

To the best of our knowledge, ODFL is the first approach that leverages adversarial compilation configurations to localize faults in GCC. In contrast, program mutation-based localization techniques (DiWi, RecBi, LLM4CBI, and ETEM) form a distinct class of approaches that requires substantial time budgets to generate and validate large numbers of mutated programs. Prior studies have demonstrated that ODFL and BASIC perform better than program mutation-based techniques in isolating compiler faults [48, 49].

Notably, ODFL relies on a single pair of adversarial configurations and performs slightly less than BASIC on GCC (20 vs. 21), while being significantly more efficient. In particular, ODFL does not require installing and testing multiple commit-level compiler versions to identify the BIC. Consequently, although we include all these approaches for completeness, our evaluation emphasizes ODFL as the most directly comparable baseline to demonstrate the effectiveness of MULTICONF's multi-pair of adversarial compilation configuration strategy.

4.1.3 Measurements. To assess the effectiveness of the compiler fault localization approaches, we utilize several measurement metrics commonly employed in existing studies [10, 11, 25, 39, 49]. These metrics allow us to evaluate the performance of the approaches from various perspectives.

— *Top-N*: This metric measures the number of bugs successfully located within the Top-N positions of the ranking list. Specifically, it assesses whether at least one faulty file appears among the Top-N files. Consistent with previous studies [10, 11, 25, 39, 49], we consider N values of 1, 5, 10, and 20. A higher value indicates better performance.
— *MFR*: MFR calculates the average rank of the first buggy element (i.e., the faulty file in our study) for each bug in the ranking list. This metric focuses on the position of the first identified buggy element, with smaller values indicating better performance.
— *MAR*: MAR calculates the average rank of all buggy elements (i.e., faulty files) for each bug in the ranking list. This metric considers the positions of all buggy elements, with smaller values indicating better performance.

4.2 RQ1: MULTICONF vs. Existing Techniques in Terms of Effectiveness

4.2.1 Experimental Settings. We adopt the same experimental setup as previous studies [10, 11, 39, 49]. Specifically, each witness program-based technique generates mutants continuously until it exceeds 1 hour of execution time. To mitigate the influence of randomness when replicating these techniques, this study directly utilizes results from prior research that employed the same set of 60 GCC bugs. This ensures a direct and equitable comparison with state-of-the-art techniques. For a fair comparison, we use the reported results of a recent study by Tu et al. under the default setting (i.e., 1-hour termination) as the effectiveness metrics for DiWi, RecBi, and LLM4CBI in our comparison with MULTICONF. For ODFL, ETEM, and BASIC, we utilize the best performing results reported in their papers.

4.2.2 Effectiveness. The results presented in Table 4 compare the effectiveness of MULTICONF with other existing compiler fault localization approaches. The row labeled "MULTICONF" shows its performance across various evaluation metrics. Overall, MULTICONF demonstrates strong bug localization capabilities. Specifically, it successfully locates 27, 40, 51, and 53 compiler bugs within

Table 4. Compiler Fault Location Effectiveness Comparison

Technique	Top-1	$\Uparrow_{Top-1}$	Top-5	$\Uparrow_{Top-5}$	Top-10	$\Uparrow_{Top-10}$	Top-20	$\Uparrow_{Top-20}$	MFR	$\Uparrow_{MFR}$	MAR	$\Uparrow_{MAR}$
MultiConf	**27.00**	—	**40.00**	—	**51.00**	—	**53.00**	—	**7.38**	—	**8.53**	—
Odfl	20.00	35.00%	31.00	29.03%	42.00	21.43%	47.00	12.77%	23.05	67.98%	23.78	64.13%
DiWi	5.60	382.14%	19.90	101.01%	31.30	62.94%	41.70	27.10%	22.57	67.30%	23.10	63.07%
RecBi	7.70	250.65%	23.90	67.36%	34.20	49.12%	42.50	24.71%	20.53	64.05%	21.06	59.50%
LLM4CBI	9.30	190.32%	24.90	60.64%	36.40	40.11%	44.80	18.30%	15.84	53.41%	16.35	47.83%
ETEM	20.00	35.00%	38.00	5.26%	44.00	15.91%	54.00	−1.85%	6.63	−11.31%	7.04	−21.16%
Basic	21.00	28.57%	35.00	14.29%	35.00	45.71%	35.00	51.43%	–	–	–	–

Columns "$\Uparrow_*$" refers to the improvement rate of MultiConf over a compared technique in terms of the * measurement. Bold values indicate the best performance among all methods for each metric.

the Top-1, Top-5, Top-10, and Top-20 files of the ranking lists, respectively. This indicates that MultiConf effectively isolates approximately 45%, 67%, 85%, and 88% of the bugs within these categories. Additionally, the MFR and MAR values for MultiConf are 7.38 and 8.53, respectively, suggesting that, on average, the first faulty file appears within the top 7.38 files and all faulty files are located within the top 8.53 files of the ranking list.

When comparing with other approaches (Odfl, DiWi, RecBi, LLM4CBI, ETEM, and Basic) in Table 4, we observe that MultiConf achieves improved effectiveness in compiler fault localization. Notably, MultiConf outperforms the other approaches in the Top-1 and Top-5 metrics, which are particularly valuable for practical applications. Specifically, compared to the state-of-the-art configuration-based approach Odfl, MultiConf achieves bug localization for 35.00%, 29.03%, 21.43%, and 12.77% more bugs within the Top-1, Top-5, Top-10, and Top-20 files, respectively. Against the recently proposed ETEM, MultiConf isolates 35.00%, 5.26%, and 15.91% more bugs within the Top-1, Top-5, and Top-10 files. Compared to Basic, a common strategy used in practice, MultiConf outperforms it by 28.57%, 14.29%, 45.71%, and 51.43% more bugs within the Top-1, Top-5, Top-10, and Top-20 files. Furthermore, MultiConf achieves the best scores in both MFR and MAR among all baselines except ETEM. These results collectively confirm that MultiConf provides more precise and accurate bug localization, establishing a new state of the art over existing approaches.

Despite these improvements, we observe that 7 out of 60 bugs (11.7%) were not successfully localized within the Top-20 files. We conducted a manual investigation of these cases and found that their localization difficulty stems primarily from fundamental limitations of SBFL techniques, rather than deficiencies specific to MultiConf. In particular, these challenging bugs exhibit one or more of the following characteristics.

— *Insufficient Coverage Differentiation*: When passing and failing configurations exhibit highly similar code coverage profiles, SBFL struggles to distinguish faulty files due to minimal differences in execution patterns.
— *Distributed Faults*: Some bugs involve complex interactions across multiple files or modules, resulting in suspiciousness scores being spread thinly across several components. This dilution makes it difficult for SBFL methods to precisely identify the main faulty location.

4.2.3 Statistical Test Analysis of Results. In alignment with previous work [39], we conduct a Mann-Whitney U-test with a significance level of 0.05 on the total number of bugs located by MultiConf versus the state-of-the-art approaches, following the guidelines of Arcuri and Briand [6]. To further mitigate the influence of randomness, we calculate the effect size of the differences between MultiConf and the baseline techniques using Vargha and Delaney's $\hat{A}_{12}$ statistics. In this context, given a performance measure M (e.g., Top-1), the $\hat{A}_{12}$ statistic quantifies the probability

Table 5. Statistic Test Results for MultiConf vs. State-of-the-Art Techniques in Terms of Locating GCC Bugs

MultiConf vs. Odfl			MultiConf vs. DiWi			MultiConf vs. RecBi		
Metric	p-value	$\hat{A}_{12}$	Metric	p-value	$\hat{A}_{12}$	Metric	p-value	$\hat{A}_{12}$
Top-1	<0.001	1.0	Top-1	<0.001	1.0	Top-1	<0.001	1.0
Top-5	<0.001	1.0	Top-5	<0.001	1.0	Top-5	<0.001	1.0
Top-10	<0.001	1.0	Top-10	<0.001	1.0	Top-10	<0.001	1.0
Top-20	<0.001	1.0	Top-20	<0.001	1.0	Top-20	<0.001	1.0
MFR	<0.001	1.0	MFR	<0.001	1.0	MFR	<0.001	1.0
MAR	<0.001	1.0	MAR	<0.001	1.0	MAR	<0.001	1.0
MultiConf vs. LLM4CBI			**MultiConf vs. ETEM**			**MultiConf vs. Basic**		
Metric	p-value	$\hat{A}_{12}$	Metric	p-value	$\hat{A}_{12}$	Metric	p-value	$\hat{A}_{12}$
Top-1	<0.001	1.0	Top-1	<0.001	1.0	Top-1	<0.001	1.0
Top-5	<0.001	1.0	Top-5	<0.001	1.0	Top-5	<0.001	1.0
Top-10	<0.001	1.0	Top-10	<0.001	1.0	Top-10	<0.001	1.0
Top-20	<0.001	1.0	Top-20	<0.001	0.0	Top-20	<0.001	1.0
MFR	<0.001	1.0	MFR	<0.001	0.0	MFR	—	—
MAR	<0.001	1.0	MAR	<0.001	0.0	MAR	—	—

that approach A (e.g., MultiConf) yields higher M values than approach B (e.g., LLM4CBI). If the two approaches are equivalent, then $\hat{A}_{12} = 0.5$. For instance, $\hat{A}_{12} = 0.9$ indicates that approach A produces higher results 90% of the time compared to approach B.

We utilize the R programming language to perform the Mann-Whitney U-test and obtain the p-value. To calculate the effect size, we employ the following formula:

$$\hat{A}_{12} = \frac{(R1/m) - \left(\frac{m+1}{2}\right)}{n}, \tag{4}$$

where $R1$ is the rank sum of the first data group being compared, and m represents the number of observations in the first group and n represents the number of observations in the second group. The rank sum is a fundamental component of the Mann-Whitney U-test and is provided by most statistical tools. We compute the $R1$ value using the function "$sum(rank(c(X, Y))[seq_along(X)])$" in R, where X and Y are the datasets containing the observations (i.e., metrics over GCC bugs) of the two approaches being compared.

The overall statistical results are presented in Table 5. A p-value less than 0.05 (often less than 0.001) indicates that MultiConf significantly outperforms all existing approaches, including Odfl [49], DiWi [10], RecBi [11], LLM4CBI [39], and ETEM [25]. Furthermore, we observe that the effect sizes are 1.0 for all techniques except ETEM in the Top-20, MFR, and MAR metrics, suggesting that MultiConf has a strong likelihood of yielding better results than existing approaches.

4.2.4 Analysis of Bugs Uniquely Isolated by MultiConf. To our best knowledge, Odfl is the only prior approach that leverages adversarial compilation configurations to localize faults in GCC compilers. In terms of both localization effectiveness and computational cost, Odfl represents the strongest existing technique for isolating GCC faults. Accordingly, we select Odfl as the primary baseline for direct comparison in our evaluation.

To assess MultiConf's unique advantages, we further analyze the bugs that can be successfully localized by MultiConf but not by Odfl. As shown in Table 6, MultiConf uniquely localizes 10

Table 6. Bugs Uniquely Isolated by MultiConf (✓: Bug Can be Isolated in Top-1; ✗: Cannot be Isolated)

No.	BugId	MultiConf	Odfl	No.	Bug ID	MultiConf	Odfl
1	56478	✓	✓	31	62151	✓	✓
2	57303			32	63551		
3	57341			33	63605		
4	57488			34	63659		
5	57521	✓	✗	35	64041		
6	57719	✓	✗	36	64682		
7	58068			37	64756	✓	✓
8	58223	✓	✗	38	64853		
9	58343			39	65318	✓	✗
10	58418			40	66186		
11	58451	✓	✗	41	66272		
12	58539			42	66375		
13	58570			43	66863		
14	58640			44	66894	✓	✓
15	58662	✗	✓	45	66952		
16	58696			46	67121		
17	58726	✗	✓	47	67456	✓	✓
18	58759			48	67786	✗	✓
19	59221			49	67828	✓	✗
20	59594			50	68194	✓	✓
21	59715	✓	✓	51	68250		
22	59747	✓	✓	52	68990	✓	✓
23	60452			53	69951	✓	✓
24	61140	✓	✓	54	70138	✓	✗
25	61306	✓	✓	55	70586	✓	✓
26	61383	✓	✓	56	71439	✓	✗
27	61517	✓	✓	57	71518		
28	61518			58	80622	✓	✓
29	61578	✓	✗	59	81740		
30	61681	✓	✗	60	82078	✓	✓

bugs that Odfl fails to identify. A closer examination of these cases reveals a consistent pattern: the truly faulty files rarely appear at the very top (Top-1) of any single ranking produced by our three adversarial configuration pairs. Instead, they tend to appear within the Top-5 positions across multiple rankings. The aggregation step in MultiConf is therefore critical—it consolidates these consistent, moderate signals from different configuration pairs, elevating the true faulty files above distractor files that only achieve high ranking in a single configuration.

We also observe that Odfl's effectiveness is comparable to our variant $AdvConf_2$, despite their different strategies for producing adversarial configurations. Specifically, $AdvConf_2$ constructs a failing configuration by directly disabling half of the bug-free options without any additional search, making it substantially more time-efficient. The additional pairs, $AdvConf_1$ and $AdvConf_3$, however, significantly improve robustness. These pairs are derived from distinct failing configurations, which increases diversity in both coverage and failure-inducing behaviors, and thereby reducing the likelihood that innocent files are consistently ranked highly. This diversity enables our aggregation step to downweight inconsistently ranked files and to amplify those that are consistently suspicious across multiple configuration pairs.

Concrete examples:

— *GCC Bug #57719*: The actual faulty file is `tree-loop-distribution.c`. All other approaches, including Odfl, consistently rank `lra-assigns.c` as Top-1. In contrast, across our three individual rankings, `tree-loop-distribution.c` appears at ranks 2, 4, and 4. Meanwhile, `lra-assigns.c` is ranked Top-1 in two configurations but does not appear in the $AdvConf_1$

Table 7. Comparison of Time Consumption in Core Generation Phase (Hours)

Approach	Average	Maximum	Minimum
MultiConf	0.0135	0.0196	0.0103
Odfl	0.0035	0.0066	0.0019
DiWi	1.0000	1.0000	1.0000
RecBi	1.0000	1.0000	1.0000
LLM4CBI	1.0000	1.0000	1.0000
ETEM	1.0000	1.0000	1.0000

ranking at all, indicating unstable suspiciousness and suggesting it is an innocent file. Through aggregation, MultiConf correctly highlights `tree-loop-distribution.c` as the most suspicious file, demonstrating how consistent mid-top rankings across diverse configuration pairs can outweigh a single spurious Top-1 result.

— *GCC Bug #70138 (also Discussed in Section 3.2.5)*: Both Odfl and $AdvConf_2$ incorrectly prioritize `tree-vect-slp.c`. Under the $AdvConf_3$ configuration, however, `tree-vect-slp.c` drops to rank 67, revealing its instability across configurations. In contrast, the true faulty file, `tree-vect-loop-manip.c`, remains consistently highly ranked (4, 3, and 4, all within the Top-5). Aggregation enables MultiConf to correctly identify the faulty file despite contradictory signals from individual configuration pairs.

These findings highlight a fundamental limitation of single-pair localization methods: suspiciousness derived from a single failing–passing pair is vulnerable to incidental coverage overlap. By integrating multiple and diverse configuration pairs, MultiConf systematically suppresses inconsistently ranked files and promotes those that repeatedly appear near the top, achieving more accurate and robust fault localization.

4.3 RQ2: MultiConf vs. Existing Techniques in Terms of Efficiency

4.3.1 Experimental Setup. To evaluate the efficiency of MultiConf, we compared its time consumption with five representative approaches: Odfl, DiWi, RecBi, LLM4CBI, and ETEM. Our measurements focus exclusively on the core generation phase for each method.

— For *program-based baselines* (DiWi, RecBi, LLM4CBI, and ETEM), we followed the original protocol and used a 1-hour timeout for program mutation. This timeout is configurable, but we adopted the standard setting for consistency.
— For *configuration-based methods* (MultiConf and Odfl), we measured the time devoted to generating adversarial compilation configurations.

All subsequent procedures—including coverage collection (via `gcov`) and SBFL analysis (using the Ochiai metric)—were performed identically for all methods and are excluded from our timing data.

4.3.2 Experimental Results. Table 7 summarizes the results. As shown, MultiConf generates three pairs of adversarial compilation configurations in an average of 0.0135 hours, a runtime that is only marginally higher than Odfl's 0.0035 hours. By contrast, each program-based baseline (DiWi, RecBi, LLM4CBI, and ETEM) consistently reaches the 1-hour timeout for program mutation.

The 1-hour time reported for the baselines reflects only the mutation phase, matching the protocols in their original papers. By excluding post-processing from the timing, we ensure that efficiency comparisons reflect only the core algorithmic steps.

Compared to Odfl, the state-of-the-art configuration-based approach in compiler fault localization, MultiConf achieves a substantial efficiency improvement, reducing configuration generation

Table 8. Comparison of MultiConf with and without Candidate Filtering

Technique	Top-1	$\Uparrow_{Top-1}$	Top-5	$\Uparrow_{Top-5}$	Top-10	$\Uparrow_{Top-10}$	Top-20	$\Uparrow_{Top-20}$	MFR	$\Uparrow_{MFR}$	MAR	$\Uparrow_{MAR}$
MultiConf	27	—	40	—	51	—	53	—	7.38	—	8.53	—
MultiConf$_{wf}$	25	8.00%	38	5.26%	49	4.08%	51	3.92%	9.52	22.48%	10.96	22.17%

Columns "$\Uparrow_*$" refers to the improvement rate of MultiConf over MultiConf$_{wf}$ in terms of the * measurement.

time by nearly 93%. This efficiency gain is attributable to MultiConf's streamlined approach for producing adversarial configurations, in contrast to Odfl's more complex search strategy designed for maximizing effectiveness per configuration pair.

4.4 RQ3: Impact of Candidate Filtering on the Effectiveness of MultiConf

4.4.1 Experimental Setup. Candidate filtering in MultiConf is designed to mitigate the expansive search space inherent to modern compilers such as GCC, which typically comprise hundreds of source files (380 C files on average in our benchmark suite). To rigorously evaluate the impact of candidate filtering on both fault localization effectiveness and computational efficiency, we conduct an ablation study under the following two configurations.

— *MultiConf*: The complete version of our approach, incorporating a two-stage candidate filtering pipeline:
  (1) *Execution Trace Filtering*: Excludes source files that are never exercised during compilation, immediately reducing the candidate set by 20% (from 380 to 304).
  (2) *Coverage Differential Filtering*: Further eliminates files whose coverage spectra are identical between passing and failing configurations, yielding an additional 57.5% reduction (from 304 to 129).

— *MultiConf$_{wf}$*: A baseline variant *w*ithout *f*iltering, which bypasses both filtering stages and analyzes all 380 files.

Both approaches were evaluated on a consistent set of 60 real-world GCC bugs, ensuring a fair and direct comparison.

4.4.2 Experimental Results.

(1) *Improved Fault Localization Accuracy*: As shown in Table 8, enabling candidate filtering increases the Top-1 accuracy of MultiConf from 25 to 27 (an 8.00% improvement) and yields consistent gains across Top-5, Top-10, and Top-20 metrics, as well as ranking-based measures (MFR and MAR).
(2) *Greater Efficiency*: As shown in Table 9, candidate filtering reduces the average analysis time by 1.51% (from 0.0464 to 0.0457 hours). Although the filtering stage introduces a modest overhead, it substantially accelerates the core SBFL computation by reducing the average number of analyzed files from 380 to 129, thereby narrowing the search space and lowering overall computational cost.

Overall, these results demonstrate that candidate filtering in MultiConf effectively balances search space reduction with localization accuracy, achieving both practical efficiency gains and improved fault localization performance.

Table 9. Compiler Fault Location Efficiency Comparison (Hours)

Approach	Average	⇓	Maximum	⇓	Minimum	⇓
MultiConf	0.0457	1.51%	0.1014	9.22%	0.0178	−6.59%
MultiConf$_{wf}$	0.0464	—	0.1117	—	0.0167	—

Columns "⇓" refer to the reduction rate of MultiConf over MultiConf$_{wf}$ in time consumption.

Table 10. Tradeoffs of Varying Pair Numbers

Technique	Top-1	⇑$_{Top-1}$	Top-5	⇑$_{Top-5}$	Top-10	⇑$_{Top-10}$	Top-20	⇑$_{Top-20}$	MFR	⇑$_{MFR}$	MAR	⇑$_{MAR}$
$AdvConf_1$	18	50.00%	34	17.65%	44	15.91%	51	3.92%	8.61	14.29%	9.92	14.01%
$AdvConf_2$	16	68.75%	33	21.21%	46	10.87%	54	−1.85%	8.73	15.46%	9.54	10.59%
$AdvConf_3$	10	170.00%	35	14.29%	46	10.87%	52	1.92%	9.78	24.54%	10.71	20.35%
MultiConf	**27**	—	**40**	—	**51**	—	53	—	7.38	—	8.53	—
MultiConf$_{5p}$	25	8.00%	**40**	0.00%	46	10.87%	54	−1.85%	**7.25**	−1.79%	**8.22**	−3.77%
MultiConf$_{10p}$	24	12.50%	39	2.56%	**51**	0.00%	**55**	−3.64%	7.37	−0.01%	8.52	−0.01%

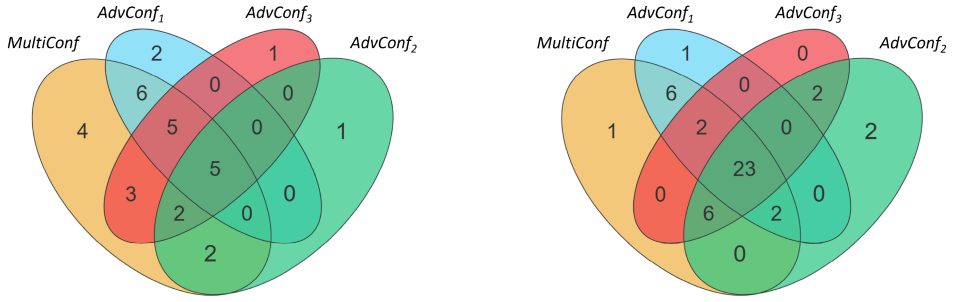
Columns "⇑$_*$" refers to the improvement rate of MultiConf over other variants in terms of the * measurement. Bold values indicate the best performance among all variants for each metric.

Table 11. Different Pair Numbers Efficiency Comparison (Hours)

Approach	Average	Maximum	Minimum
MultiConf$_{1p}$	0.0214	0.0412	0.0079
MultiConf	0.0457	0.1014	0.0178
MultiConf$_{5p}$	0.0689	0.1601	0.0244
MultiConf$_{10p}$	0.1286	0.3064	0.0350

4.5 RQ4: Impact of the Number of Adversarial Compilation Configuration Pairs on MultiConf's Effectiveness

4.5.1 Experimental Setup. To investigate the effect of the number of adversarial configuration pairs, we systematically varied the number of pairs used by MultiConf (i.e., 1, 3, 5, and 10) and analyzed the resulting tradeoffs in accuracy and efficiency. For the single-pair scenario, we introduced three variants, each corresponding to a different adversarial configuration pair and omitting the rank aggregation step, denoted as $AdvConf_1$, $AdvConf_2$, and $AdvConf_3$ (detailed in Section 3.2.3). This design allows us to isolate the benefits of rank aggregation in MultiConf. Table 10 presents a comparative analysis of these variants and MultiConf. We further examine the distribution of correctly localized bugs using the Top-1 and Top-5 metrics. For experiments with three adversarial pairs, the full MultiConf algorithm is employed. The three configurations respectively disable all, half, and none of the bug-free options. For five pairs, we add two more configurations that disable a quarter and three-quarters of the bug-free options. In the 10-pair setting, configurations are distributed more finely: 1 disables all, 1 disables none, and the remaining 8 disable increments from one-eighth to seven-eighths of the bug-free options. This setup enables a comprehensive assessment of MultiConf across a diverse spectrum of adversarial scenarios. The results, including localization effectiveness and the impact of rank aggregation, are summarized in Table 10. Efficiency results, in terms of average, maximum, and minimum running time, are provided in Table 11.



(a) Top-1 (b) Top-5

Fig. 3. Distribution of correct localization achieved by MultiConf and individual pairs (Top-1 and Top-5).

4.5.2 Experimental Results. Our results show that different adversarial pairs have varying abilities to localize compiler bugs, supporting the rationale for using rank aggregation in MultiConf. As illustrated in Figure 3, some bugs are uniquely localized by MultiConf when aggregating across pairs, while many bugs identified by individual pairs are also captured by MultiConf.

Table 10 demonstrates that MultiConf consistently outperforms single-pair variants, especially in the Top-1 metric, achieving improvements of 50%, 68.75%, and 170% over $AdvConf_1$, $AdvConf_2$, and $AdvConf_3$, respectively. This highlights the effectiveness of rank aggregation in combining the strengths of multiple adversarial configurations for more accurate bug localization.

Interestingly, Figure 3(a) shows that 4 out of 27 bugs localized by MultiConf in the Top-1 setting are not found by any individual pair, illustrating the complementary power of aggregation. Similar patterns are observed for Top-5 in Figure 3(b).

We further investigate the impact of increasing the number of adversarial configuration pairs. As shown in Tables 10 and 11, our results reveal that three pairs offer the best balance between accuracy and efficiency.

- *Saturation Effect*: Accuracy plateaus at three pairs (27 Top-1 in Table 10), with marginal or no gains beyond this point. Specifically, while increasing from one pair to three pairs brings significant improvements across all metrics—for example, Top-1 rises from 18 to 27 (a 50% increase), and MFR drops from 8.61 to 7.38—using more than 3 pairs does not yield further substantial gains. When 5 pairs are used, Top-1 actually drops to 25, and Top-5 remains at 40, identical to the three-pair configuration. With ten pairs, Top-1 further decreases to 24, and gains in other metrics are either negligible or negative.
- *Efficiency Tradeoff*: Increasing the number of adversarial pairs has a notable impact on computational efficiency. The runtime for the three-pair configuration is 0.0457 hours on average, but this increases threefold to 0.1286 hours when using ten pairs, as shown in Table 11. The computational overhead thus scales much faster than accuracy improvements, leading to an inefficient use of resources at higher pair counts.

While generating more pairs is technically feasible, our empirical evidence indicates that using three adversarial pairs achieves near-optimal accuracy with minimal computational overhead. Increasing the number further yields diminishing returns and higher costs, making three pairs the recommended configuration for practical use.

4.6 RQ5: Effect of Different SBFL Formulae in MultiConf

4.6.1 Experimental Setup. MultiConf defaults to using the Ochiai formula [2] to calculate the suspiciousness value of statements in the compiler. This choice is based on previous reports that

Table 12. The Six Adapted SBFL Formulae in MultiConf

Name	Formula
Ochiai [2]	$\frac{ef_s}{\sqrt{(ef_s + nf_s)(ef_s + ep_s)}}$
Tarantula [20]	$\frac{ef_s/totalf_s}{ef_s/totalf_s + ep_s/totalp_s}$
DStar [43]	$\frac{ef_s^\star}{ep_s + nf_s}$
Dice [44]	$\frac{2ef_s}{ef_s + nf_s + ep_s}$
Barinel [4]	$1 - \frac{ep_s}{ep_s + ef_s}$
Op2 [29]	$ef_s - \frac{ep_s}{(totalp_s + 1)}$

We used $\star = 2$, the most thoroughly explored value [32].

have identified Ochiai as the most effective formula for SBFL [32, 44]. However, it remains unknown whether MultiConf can achieve similar performance using alternative SBFL formulae. To explore the impact of different formula choices on the performance of MultiConf, we compare Ochiai with five other most widely studied SBFL formulae [32, 41, 44]. This comparative analysis aims to offer insights into the effectiveness of various formulae within the context of MultiConf and its ability to accurately localize compiler bugs. An overview of these SBFL formulae is provided in Table 12. In these formulae, ef_s, nf_s, and ep_s are defined in the same way as explained in Formula (1), np_s denotes the number of passing cases that do not execute the statement s, and $totalf_s$ and $totalp_s$ denote the total number of failing cases and passing cases for statement s respectively. We studied how different formulae affect the performance of MultiConf and evaluated them with Top-1, Top-5, Top-10, Top-20, MAR, and MFR.

4.6.2 Experimental Results. Table 13 presents the results of MultiConf using different formulae. In this table, different rows represent the different SBFL formulae utilized, and different columns represent the different metrics evaluated. It can be observed that MultiConf achieves almost similar results when adopting the formulae Ochiai, Tarantula, Dstar, Dice and Barinel. This explains why most of the formulae have no obvious effect on the performance of MultiConf and reveals the effectiveness and stability of MultiConf. However, all these formulae are not specifically designed for locating compiler bugs, and whether more effective formulae could be utilized remains unknown. We leave such exploration as our future work.

5 Discussion

5.1 Limitations and Scope

MultiConf is specifically designed to localize the faulty files associated with option-sensitive compiler bugs—those whose manifestation is influenced by the selection of compiler options,

Table 13. Performance of MultiConf Under Different SBFL Formulae

Formula	Top-1	Top-5	Top-10	Top-20	MAR	MFR
Ochiai	**27**	40	**51**	**53**	7.38	8.53
Tarantula	22	**41**	50	**53**	7.73	8.81
DStar	22	38	44	48	10.77	11.78
Dice	25	**41**	48	**53**	**7.28**	**8.30**
Barinel	25	**41**	48	**53**	**7.28**	8.31
Op2	22	38	49	52	8.27	9.23

Bold indicates the best result for each metric.

particularly optimization flags. According to the official GCC bug taxonomy,[1] our approach is most applicable to the following primary categories.

— *Wrong Code*: Bugs where the compiler produces incorrect output code. These issues frequently depend on particular optimization or configuration settings. By analyzing behavioral differences under various options, our technique can help pinpoint the source files most likely responsible for such failures.
— *ICE on Valid Code*: ICEs or assertion failures encountered when compiling valid source code. These are often triggered by specific combinations of compiler flags, and our approach is effective in localizing the files at fault in these scenarios.

There are several limitations to our technique.

— *Option Independence*: MultiConf is not intended for bugs whose manifestation is independent of compiler options; such bugs fall outside the scope of our localization strategy.
— *Option Coverage*: Accuracy may be reduced for bugs triggered only by rare or highly specific option combinations not included in our experimental setup.
— *Compiler Specificity*: Our evaluation focuses on GCC. Adapting the method to compilers with different optimization infrastructures (e.g., LLVM) may require additional work.

5.2 Witness Programs from Other Compilers

Recent work has shown that real-world programs from bug reports (even from other compilers), as opposed to artificially constructed or randomly generated tests, can significantly enrich compiler testing [52]. These real-world programs are typically more representative of the practical usage scenarios and often trigger subtler, harder-to-find compiler defects. They can also be treated as witness programs in a fault localization scenario.

However, incorporating real-world witness programs into a localization framework presents notable challenges. The effectiveness of the SBFL relies on discriminative coverage information. Existing witness-based approaches typically minimize code-coverage differences between bug-triggering and witness programs with only minor variations, thereby facilitating the identification of suspicious compiler modules responsible for a fault. If additional witness programs are not carefully selected, especially when sourced from other compilers or unrelated bug reports, the resulting coverage data may become noisy, making it harder to distinguish faulty components from non-faulty ones. This risk is especially pronounced in large, complex compiler codebases where even minor differences in exercised code paths can significantly affect localization outcomes.

[1] https://gcc.gnu.org/bugzilla/describekeywords.cgi.

Therefore, while integrating real-world seeds holds promise for expanding test coverage and bug detection, their indiscriminate use in fault localization must be approached with caution. A promising direction for future work is the development of hybrid strategies that combine the strengths of both targeted witness programs and adversarial configuration pairs, balancing increased robustness with precise localization.

5.3 Relation to Program Reduction

Program reduction techniques, such as hierarchical delta debugging [26], probabilistic delta debugging [40], cause reduction [17], and C-Reduce [33], play a pivotal role in the testing and debugging of compilers and other complex software systems [37]. Their primary objective is to shrink complex, failure-inducing test cases to minimal programs that still trigger the same compiler bug. This minimization improves human readability, facilitates bug reproduction, and streamlines manual analysis.

These reduction methods directly complement compiler fault localization techniques like MultiConf. Once a test case has been reduced, the compiler is exercised on a smaller, more focused input, causing only a subset of its codebase to be executed. This has two main benefits for localization: (1) it eliminates noise by excluding compiler code unrelated to the bug, and (2) it increases the precision of coverage-based analyses, as the remaining execution paths are more likely to be causally linked to the failure. As a result, MultiConf can leverage these cleaner, more targeted traces to pinpoint the underlying faulty compiler components more accurately and efficiently.

While program reduction and fault localization address different stages of the debugging workflow, they are highly synergistic: reduction enhances the effectiveness of localization but does not replace the need for specialized techniques to map failures in test programs to faulty source code within the compiler.

5.4 Threats to Validity

5.4.1 *Internal Validity.*

(1) *Configuration Selection Bias*: Although our systematic approach to generating adversarial configurations (Section 3.2.3) reduces arbitrary selection, results may still be influenced by the specific configurations chosen. We address this by utilizing multiple configuration pairs to diversify coverage and by applying a weighted aggregation mechanism to further limit dependence on any single configuration pair.
(2) *SBFL Formula Sensitivity*: As shown in our experiments (RQ5), Top-1 accuracy varies by less than 5% across 6 SBFL formulae (Ochiai, Tarantula, etc.), suggesting our method is robust to formula selection. We select Ochiai as the default due to its consistently strong performance.
(3) *Impact of Filtering*: The candidate filtering step reduces the search space from 380 to 129 files, improving efficiency. However, in rare cases where coverage differences are subtle, this process may inadvertently exclude relevant files. While such cases did not occur in our benchmark, further investigation is needed to assess their frequency and impact in broader settings.
(4) *Randomness in Baseline Comparisons*: Some baseline approaches (e.g., DiWi, RecBi, LLM4CBI, ETEM) involve randomized mutations, introducing variance in fault localization outcomes. To control for this, we report results from prior controlled experiments, minimizing the influence of randomness on comparative analysis.

5.4.2 *External Validity.*

(1) *Compiler Specificity*: Our approach is tailored to GCC's optimization infrastructure and does not directly generalize to other ordered-pass-based compilers like LLVM (see Section 4.1.1). Adapting our methodology to different architectures remains future work.

(2) *Benchmark Size*: While our evaluation on 60 bugs matches the scale of prior work, this sample may not fully capture the diversity of real-world scenarios. Expanding our benchmark set and including a broader range of bug types are important for strengthening external validity.

5.5 Future Work

Several promising directions remain for extending this research.

(1) *Minimal Trigger Set Identification*: At present, when no individual bug-triggering options can be isolated, we generate adversarial configuration pairs at the optimization level granularity (see Section 3.2.3). However, to more effectively address complex interactions among optimization options, a promising direction for future work is to identify minimal sets of options that collectively cause the bug. This approach could yield more precise insights than operating solely at the optimization level granularity.
(2) *Hybrid with Witnesses*: MultiConf is orthogonal to witness-program-based approaches. Combining a carefully selected set of real or witness programs with multiple adversarial configuration pairs may further enhance the robustness and effectiveness of fault localization.
(3) *Live Bug Evaluation*: Beyond benchmarked bugs, applying MultiConf to current, unresolved compiler issues and engaging with developers to obtain feedback would provide stronger evidence for the practical utility and real-world impact of our method.
(4) *Benchmark Expansion*: To improve external validity, we intend to expand our benchmark suite, including more bugs and a wider variety of bug types for a more comprehensive evaluation.
(5) *Cross-Compiler Generalization*: We intend to adapt our approach to other compilers, such as LLVM, by evolving our methodology to accommodate architectural differences in optimization pipelines and pass management.

These future directions aim to address current limitations, strengthen the validity of our results, and broaden the impact of our approach in real-world compiler development and maintenance.

6 Related Work

Software fault localization has been extensively studied, resulting in a rich body of literature encompassing a wide range of techniques. Representative approaches include SBFL [4, 38, 41], slicing-based techniques [5, 51], mutation-based approaches [18, 27, 31], and model-based fault localization [3, 42]. Among these, SBFL techniques [4, 38] have received the most research attention, as they quantify the suspiciousness of program elements by contrasting their execution coverage in failing and passing test cases.

Compiler fault localization introduces additional challenges due to the complexity of compilers and the difficulty of constructing suitable test cases. Witness program-based approaches, such as DiWi [10], RecBi [11], LLM4CBI [39], and ETEM [25], attempt to address challenges by generating both passing and failing test programs to isolate compiler faults. These approaches leverage SBFL to rank suspicious compiler files. However, they often suffer from instability in test generation, high computational costs, and limited diversity in the generated witness programs.

Odfl [49] proposed adversarial compilation configurations as an alternative to witness program generation. By selectively disabling fine-grained compiler optimization options, Odfl constructs passing and failing configurations for SBFL-based analysis. Although effective, Odfl relies on a single adversarial configuration pair, which constrains coverage diversity and leads to suboptimal localization performance.

Basic [48] revisited compiler fault localization from the perspective of practical debugging workflows by directly identifying the BIC and treating the files modified in that commit as faulty

candidates. Its evaluation demonstrated that Basic achieves performance comparable to, and in many cases better than, state-of-the-art SBFL-based compiler fault localization techniques. Nevertheless, Basic incurs substantial computational overhead, as it requires installing and testing multiple commit-level compiler versions to identify the BIC.

To address these limitations, we propose MultiConf, a novel approach that generates multiple pairs of adversarial compilation configurations. By aggregating rankings from these configuration pairs, MultiConf substantially increases coverage diversity and improves localization accuracy. This design is inspired by ensemble learning, in which combining diverse predictors mitigates individual bias [54, 55]. Our empirical evaluation on real-world GCC bugs demonstrates that MultiConf consistently outperforms existing compiler fault isolation techniques.

7 Conclusion

In this study, we introduce MultiConf, a novel approach for isolating compiler faults by leveraging multiple pairs of adversarial compilation configurations. By aggregating ranks from diverse configuration pairs, MultiConf represents a significant advancement in fault localization accuracy. Our experimental results demonstrate the substantial improvements of MultiConf over existing approaches in both effectiveness and efficiency. Specifically, MultiConf successfully localizes 27 out of the 60 GCC bugs within the Top-1 file, which is critically important for practical applications, reflecting an impressive 35.0% improvement over Odfl. The artifact of this work is publicly available at: https://zenodo.org/records/18051063.

Acknowledgments

We are grateful to the anonymous reviewers and the editorial team for their valuable and constructive feedback. We would like to express our appreciation to the authors of DiWi, RecBi, LLM4CBI, Odfl, and Basic for generously sharing their tools and data.

References

[1] GNU. 2023. GCC, the GNU Compiler Collection. Retrieved from https://gcc.gnu.org/

[2] Rui Abreu, Peter Zoeteweij, and Arjan J. C. Van Gemund. 2007. On the accuracy of spectrum-based fault localization. In *Proceedings of the Testing: Academic and Industrial Conference Practice and Research techniques-MUTATION (TAICPART-MUTATION '07)*, 89–98.

[3] Rui Abreu, Peter Zoeteweij, and Arjan J. C. van Gemund. 2008. An observation-based model for fault localization. In *Proceedings of the 2008 International Workshop on Dynamic Analysis: Held in conjunction with the ACM SIGSOFT International Symposium on Software Testing and Analysis (WODA '08/ISSTA '08)*, 64–70.

[4] Rui Abreu, Peter Zoeteweij, and Arjan J. C. van Gemund. 2009. Spectrum-based multiple fault localization. In *Proceedings of the 24th IEEE/ACM International Conference on Automated Software Engineering (ASE '09)*, 88–99.

[5] Hiralal Agrawal, Richard A. DeMillo, and Eugene H. Spafford. 1993. Debugging with dynamic slicing and backtracking. *Software: Practice and Experience* 23, 6 (1993), 589–616.

[6] Andrea Arcuri and Lionel Briand. 2011. A practical guide for using statistical tests to assess randomized algorithms in software engineering. In *Proceedings of the 33rd International Conference on Software Engineering (ICSE '11)*, 1–10. DOI: https://doi.org/10.1145/1985793.1985795

[7] Gergö Barany. 2018. Finding missed compiler optimizations by differential testing. In *Proceedings of the 27th International Conference on Compiler Construction (CC '18)*. ACM, New York, NY, 82–92. DOI: https://doi.org/10.1145/3178372.3179521

[8] Daniel Berlin, David Edelsohn, and Sebastian Pop. 2004. High-level loop optimizations for GCC. In *Proceedings of the 2004 GCC Developers Summit*, 37–54.

[9] Leslie Pérez Cáceres, Federico Pagnozzi, Alberto Franzin, and Thomas Stützle. 2017. Automatic configuration of GCC using irace. In *Proceedings of the 13th International Conference on Évolution Artificielle (EA '17)*, 202–216.

[10] Junjie Chen, Jiaqi Han, Peiyi Sun, Lingming Zhang, Dan Hao, and Lu Zhang. 2019. Compiler bug isolation via effective witness test program generation. In *Proceedings of the ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/SIGSOFT FSE '19)*, 223–234.

[11] Junjie Chen, Haoyang Ma, and Lingming Zhang. 2020. Enhanced compiler bug isolation via memoized search. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE '20)*, 78–89.

[12] Junjie Chen, Jibesh Patra, Michael Pradel, Yingfei Xiong, Hongyu Zhang, Dan Hao, and Lu Zhang. 2020. A survey of compiler testing. *ACM Computing Surveys* 53, 1, Article 4 (Jan. 2020), 36 pages.

[13] Junjie Chen and Chenyao Suo. 2022. Boosting compiler testing via compiler optimization exploration. *ACM Transactions on Software Engineering and Methodology* 31, 4, Article 72 (Aug. 2022), 33 pages. DOI: https://doi.org/10.1145/3508362

[14] Grigori Fursin, Cupertino Miranda, Olivier Temam, Mircea Namolaru, Elad Yom-Tov, Ayal Zaks, Bilha Mendelson, Edwin Bonilla, John Thomson, Hugh Leather, et al. 2008. MILEPOST GCC: Machine learning based research compiler. In *Proceedings of the GCC Summit*.

[15] Hongyan Gao, Yibiao Yang, Maolin Sun, Jiangchang Wu, Yuming Zhou, and Baowen Xu. 2025. Clozemaster: Fuzzing rust compiler by harnessing LLMs for infilling masked real programs. In *Proceedings of the 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 1422–1435. DOI: https://doi.org/10.1109/ICSE55347.2025.00175

[16] Taras Glek and Jan, Hubicka. 2010. Optimizing real world applications with GCC link time optimization. arXiv:1010.2196. Retrieved from https://arxiv.org/abs/1010.2196

[17] Alex Groce, Mohammad Amin Alipour, Chaoqiang Zhang, Yang Chen, and John Regehr. 2016. Cause reduction: Delta debugging, even without bugs. *Software Testing, Verification and Reliability* 26, 1 (Jan. 2016), 40–68. DOI: https://doi.org/10.1002/stvr.1574

[18] Shin Hong, Byeongcheol Lee, Taehoon Kwak, Yiru Jeon, Bongsuk Ko, Yunho Kim, and Moonzoo Kim. 2015. Mutation-based fault localization for real-world multilingual programs (T). In *Proceedings of the 30th IEEE/ACM International Conference on Automated Software Engineering (ASE '15)*, 464–475.

[19] Haoxiang Jia, Ming Wen, Zifan Xie, Xiaochen Guo, Rongxin Wu, Maolin Sun, Kang Chen, and Hai Jin. 2023. Detecting JVM JIT compiler bugs via exploring two-dimensional input spaces. In *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 43–55. DOI: https://doi.org/10.1109/ICSE48619.2023.00016

[20] James A. Jones and Mary Jean Harrold. 2005. Empirical evaluation of the tarantula automatic fault-localization technique. In *Proceedings of the 20th IEEE/ACM International Conference on Automated Software Engineering (ASE '05)*, 273–282.

[21] Fabian Keller, Lars Grunske, Simon Heiden, Antonio Filieri, André van Hoorn, and David Lo. 2017. A critical evaluation of spectrum-based fault localization techniques on a large-scale software system. In *Proceedings of the 2017 IEEE International Conference on Software Quality, Reliability and Security (QRS '17)*, 114–125.

[22] Vu Le, Mehrdad Afshari, and Zhendong Su. 2014. Compiler validation via equivalence modulo inputs. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '14)*. ACM, New York, NY, 216–226. DOI: https://doi.org/10.1145/2594291.2594334

[23] Vu Le, Chengnian Sun, and Zhendong Su. 2015. Finding deep compiler bugs via guided stochastic program mutation. *ACM SIGPLAN Notices* 50, 10 (Oct. 2015), 386–399. DOI: https://doi.org/10.1145/2858965.2814319

[24] Cong Li, Yanyan Jiang, Chang Xu, and Zhendong Su. 2023. Validating JIT compilers via compilation space exploration. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*. ACM, New York, NY, 66–79. DOI: https://doi.org/10.1145/3600006.3613140

[25] Yujie Liu, Mingxuan Zhu, Jinhao Dong, Junzhe Yu, and Dan Hao. 2024. Compiler bug isolation via enhanced test program mutation. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE '24)*. ACM, New York, NY, 819–830. DOI: https://doi.org/10.1145/3691620.3695074

[26] Ghassan Misherghi and Zhendong Su. 2006. HDD: Hierarchical delta debugging. In *Proceedings of the 28th International Conference on Software Engineering (ICSE '06)*. ACM, New York, NY, 142–151. DOI: https://doi.org/10.1145/1134285.1134307

[27] Seokhyeon Moon, Yunho Kim, Moonzoo Kim, and Shin Yoo. 2014. Ask the mutants: Mutating faulty programs for fault localization. In *Proceedings of the 7th IEEE International Conference on Software Testing, Verification and Validation (ICST '14)*, 153–162.

[28] Robin Morisset, Pankaj Pawan, and Francesco Zappa Nardelli. 2013. Compiler testing via a theory of sound optimisations in the C11/C++11 memory model. *ACM SIGPLAN Notices* 48, 6 (Jun. 2013), 187–196. DOI: https://doi.org/10.1145/2499370.2491967

[29] Lee Naish, Hua Jie Lee, and Kotagiri Ramamohanarao. 2011. A model for spectra-based software diagnosis. *ACM Transactions on Software Engineering and Methodology* 20, 3, Article 11 (2011), 1–32.

[30] Georg Ofenbeck, Tiark Rompf, and Markus Püschel. 2016. RandIR: Differential testing for embedded compilers. In *Proceedings of the 2016 7th ACM SIGPLAN Symposium on Scala (SCALA '16)*. ACM, New York, NY, 21–30. DOI: https://doi.org/10.1145/2998392.2998397

[31] Mike Papadakis and Yves Le Traon. 2015. Metallaxis-FL: Mutation-based fault localization. *Softw. Software Testing, Verification & Reliability* 25, 5–7 (2015), 605–628.

[32] Spencer Pearson, José Campos, René Just, Gordon Fraser, Rui Abreu, Michael D. Ernst, Deric Pang, and Benjamin Keller. 2017. Evaluating and improving fault localization. In *Proceedings of the 39th International Conference on Software Engineering (ICSE '17)*, 609–620.

[33] John Regehr, Yang Chen, Pascal Cuoq, Eric Eide, Chucky Ellison, and Xuejun Yang. 2012. Test-case reduction for C compiler bugs. In *Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '12)*. ACM, New York, NY, 335–346. DOI: https://doi.org/10.1145/2254064.2254104

[34] Flash Sheridan. 2007. Practical testing of a C99 compiler using output comparison. *Software: Practice and Experience* 37, 14 (Nov. 2007), 1475–1488.

[35] Chengnian Sun, Vu Le, and Zhendong Su. 2016. Finding and analyzing compiler warning defects. In *Proceedings of the 38th International Conference on Software Engineering (ICSE '16)*. ACM, New York, NY, 203–213. DOI: https://doi.org/10.1145/2884781.2884879

[36] Chengnian Sun, Vu Le, Qirun Zhang, and Zhendong Su. 2016. Toward understanding compiler bugs in GCC and LLVM. In *Proceedings of the 25th International Symposium on Software Testing and Analysis (ISSTA '16)*. ACM, New York, NY, 294–305. DOI: https://doi.org/10.1145/2931037.2931074

[37] Maolin Sun, Yibiao Yang, Jiangchang Wu, and Yuming Zhou. 2025. Validating SMT rewriters via rewrite space exploration supported by generative equality saturation. *Proceedings of the ACM on Programming Languages* 9, OOPSLA2, Article 315 (Oct. 2025), 27 pages. DOI: https://doi.org/10.1145/3763093

[38] Chung Man Tang, W. K. Chan, Yuen-Tak Yu, and Zhenyu Zhang. 2017. Accuracy graphs of spectrum-based fault localization formulas. *IEEE Transactions on Reliability* 66, 2 (2017), 403–424.

[39] Haoxin Tu, Zhide Zhou, He Jiang, Imam Nur Bani Yusuf, Yuxian Li, and Lingxiao Jiang. 2024. Isolating compiler bugs by generating effective witness programs with large language models. *IEEE Transactions on Software Engineering* 50, 7 (May 2024), 1768–1788. DOI: https://doi.org/10.1109/TSE.2024.3397822

[40] Guancheng Wang, Ruobing Shen, Junjie Chen, Yingfei Xiong, and Lu Zhang. 2021. Probabilistic delta debugging. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '21)*. ACM, New York, NY, 881–892. DOI: https://doi.org/10.1145/3468264.3468625

[41] Ming Wen, Junjie Chen, Yongqiang Tian, Rongxin Wu, Dan Hao, Shi Han, and Shing-Chi Cheung. 2021. Historical spectrum based fault localization. *IEEE Transactions on Software Engineering* 47, 11 (2021), 2348–2368.

[42] Ming Wen, Rongxin Wu, and Shing-Chi Cheung. 2016. Locus: Locating bugs from software changes. In *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering (ASE '16)*, 262–273.

[43] W. Eric Wong, Vidroha Debroy, Ruizhi Gao, and Yihao Li. 2014. The DStar method for effective software fault localization. *IEEE Transactions on Reliability* 63, 1 (2014), 290–308.

[44] W. Eric Wong, Ruizhi Gao, Yihao Li, Rui Abreu, and Franz Wotawa. 2016. A survey on software fault localization. *IEEE Transactions on Software Engineering* 42, 8 (2016), 707–740.

[45] Jiangchang Wu, Yibiao Yang, Maolin Sun, and Yuming Zhou. 2025. Unveiling compiler faults via attribute-guided compilation space exploration. In *Proceedings of the 2025 USENIX Annual Technical Conference (USENIX ATC '25)*. USENIX Association, 1109–1125. Retrieved from https://www.usenix.org/conference/atc25/presentation/wu-jiangchang

[46] Xiaoyuan Xie, Tsong Yueh Chen, Fei-Ching Kuo, and Baowen Xu. 2013. A theoretical analysis of the risk evaluation formulas for spectrum-based fault localization. *ACM Transactions on Software Engineering and Methodology* 22, 4, Article 31 (Oct. 2013), 40 pages. DOI: https://doi.org/10.1145/2522920.2522924

[47] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and understanding bugs in C compilers. *ACM SIGPLAN Notices* 46, 6 (Jun. 2011), 283–294. DOI: https://doi.org/10.1145/1993316.1993532

[48] Yibiao Yang, Qingyang Li, Maolin Sun, Jiangchang Wu, and Yuming Zhou. 2026. Using a sledgehammer to crack a nut? Revisiting automated compiler fault isolation. In *Proceedings of the 2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE '26)*. ACM, New York, NY, USA. DOI: https://doi.org/10.1145/3744916.3773254

[49] Yibiao Yang, Qingyang Li, Maolin Sun, Jing Yang, Jiangchang Wu, and Yuming Zhou. 2025. Isolating compiler faults through differentiated compilation configurations. *IEEE Transactions on Software Engineering* 51, 6 (2025), 1838–1853. DOI: https://doi.org/10.1109/TSE.2025.3569530

[50] Yibiao Yang, Maolin Sun, Jiangchang Wu, Qingyang Li, and Yuming Zhou. 2025. Debugger toolchain validation via cross-level debugging. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1 (ASPLOS '25)*. ACM, New York, NY, USA, 280–294. DOI: https://doi.org/10.1145/3669940.3707271

[51] Xiangyu Zhang, Haifeng He, Neelam Gupta, and Rajiv Gupta. 2005. Experimental evaluation of using dynamic slices for fault location. In *Proceedings of the 6th International Workshop on Automated Debugging (AADEBUG '05)*, 33–42.

[52] Hao Zhong. 2023. Enriching compiler testing with real program from bug report. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE '22)*. ACM, New York, NY, Article 40, 12 pages. DOI: https://doi.org/10.1145/3551349.3556894

[53] Zhide Zhou, Zhilei Ren, Guojun Gao, and He Jiang. 2021. An empirical study of optimization bugs in GCC and LLVM. *Journal of Systems and Software* 174 (2021), 110884.
[54] Zhi-Hua Zhou. 2025. *Ensemble Methods: foundations and Algorithms*. CRC Press.
[55] Zhi-Hua Zhou, Jianxin Wu, and Wei Tang. 2002. Ensembling neural networks: Many could be better than all. *Artificial Intelligence* 137, 1 (2002), 239–263. DOI: https://doi.org/10.1016/S0004-3702(02)00190-X
[56] Mingxuan Zhu and Dan Hao. 2023. Compiler auto-tuning via critical flag selection. In *Proceedings of the 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 1000–1011. DOI: https://doi.org/10.1109/ASE56229.2023.00209

Received 25 December 2024; revised 7 September 2025; accepted 19 November 2025